

Reinforcement Learning Summative Assignment Report

Student Name: Cedric Izabayo

Video Recording: <https://www.youtube.com/watch?v=NW5NUroN7Bw>

GitHub Repository: https://github.com/izabayo7/cedric_izabayo_rl_summative

1. Project Overview

This project implements a reinforcement learning-based adaptive exercise sequencer for Brainiacs AI, an AI coding tutor designed for beginner programmers. The core problem is determining the optimal sequence of coding exercises to present to a student, considering their current knowledge levels across five programming topics, their engagement level, and the appropriate difficulty and support for each exercise. Three RL algorithms were implemented and compared: Deep Q-Network (DQN), REINFORCE (a Monte Carlo policy gradient method), and Proximal Policy Optimization (PPO). Each agent was trained on a custom Gymnasium environment that simulates a beginner student learning to code, with the goal of maximizing knowledge acquisition across all topics while preventing student frustration and dropout. DQN emerged as the best-performing method, achieving a mean reward of 832.52 and consistently raising all five knowledge areas to 0.55–0.74 while maintaining student engagement above 0.70 across 200-step sessions.

2. Environment Description

a. Agent(s)

The agent represents an AI tutor that decides what coding exercise to present to a simulated beginner programmer at each time step. The agent observes the student's current state—including knowledge levels across five programming topics, engagement level, recent performance streaks, and session progress—and selects an action that specifies three aspects of the next exercise: the topic to cover, the difficulty level, and the amount of support to provide. The agent's goal is to learn a teaching policy that maximizes overall student learning (raising all five knowledge areas toward mastery) while keeping the student engaged and preventing dropout. The agent does not directly control the student's responses; instead, it must learn to anticipate how its exercise choices will affect a stochastic simulated student whose probability of answering correctly depends on their current knowledge, the exercise difficulty, and the support provided.

b. Action Space

The action space is discrete with 45 possible actions, representing every combination of three dimensions:

- Topic (5 choices): `variable_scope`, `off_by_one`, `type_mismatch`, `loop_logic`, `function_params`
- Difficulty (3 choices): `easy`, `medium`, `hard`
- Support (3 choices): `no_support`, `hint`, `worked_example`

Actions are encoded as a single integer: $\text{action} = \text{topic} \times 9 + \text{difficulty} \times 3 + \text{support}$, yielding $5 \times 3 \times 3 = 45$ discrete actions. Decoding uses: $\text{topic} = \text{action} // 9$, $\text{difficulty} = (\text{action} \% 9) // 3$, $\text{support} = \text{action} \% 3$.

c. Observation Space

The observation space is a continuous Box(13,) with all values normalized to [0, 1]. The 13 features are:

Index	Feature	Description
0–4	Knowledge levels	Mastery of each of the 5 topics (0.0 to 1.0)
5	Engagement	Student motivation level (decays with frustration)
6	Consecutive correct	Streak of correct answers (divided by 10, capped at 1.0)
7	Consecutive incorrect	Streak of incorrect answers (divided by 10, capped at 1.0)
8	Last topic	Previous topic index normalized (0–4 mapped to 0–1)
9	Last difficulty	Previous difficulty normalized (0–2 mapped to 0–1)
10	Session progress	step_count / max_steps (fraction of session elapsed)
11	Avg knowledge	Mean of all 5 knowledge values
12	Knowledge variance	Variance across 5 topics (indicates imbalance)

This observation provides the agent with a partial view of the student state—enough to make informed tutoring decisions without revealing the full internal simulation mechanics. The agent must learn to interpret these signals to infer what the student needs next.

d. Reward Structure

The reward function is designed to incentivize effective, balanced tutoring. It combines several components:

Correct answer rewards: +3.0 if the student’s knowledge in that topic is below 0.5 (rewarding progress in weak areas), +1.0 otherwise. A zone of proximal development bonus of +1.5 is added when the exercise difficulty closely matches the student’s current ability ($|\text{knowledge} - \text{difficulty_threshold}| < 0.3$). A penalty of –1.0 is applied for giving trivially easy exercises to students who already know the material (difficulty=easy when knowledge > 0.7).

Incorrect answer penalties: –0.5 base penalty. An additional –2.0 if a hard exercise was given to a student with knowledge below 0.3 (punishing poor pedagogical choices).

Balanced teaching incentives: +2.0 for teaching the weakest topic. –3.0 if more than 40% of all exercises have been on a single topic. $-5.0 \times \text{knowledge_variance}$ to penalize imbalanced knowledge. $+2.0 \times \min_knowledge$ to reward raising the weakest area.

Engagement and mastery: $+0.5 \times \text{engagement}$ (incentivizes keeping the student motivated). –10.0 for student dropout (engagement reaches 0). +0.5 per topic above 0.5, +1.5 per topic above 0.8, and +20.0 for achieving full mastery (all 5 topics ≥ 0.8).

Formal Reward Formulation:

$$R(t) = (w_1 \cdot I_{\text{correct}} \cdot K_{\text{bonus}}) + (w_2 \cdot B_{\text{zpd}}) - (w_3 \cdot P_{\text{easy}}) - (w_4 \cdot \text{Var}(K)) + (w_5 \cdot \min(K)) + (w_6 \cdot E) - P_{\text{dropout}} + B_{\text{mastery}}$$

- **K_{bonus}** : Higher reward for progress in weak areas ($K < 0.5$)
- **B_{zpd}** : Zone of Proximal Development bonus (+1.5 when difficulty matches ability)
- **P_{easy}** : Penalty for trivially easy exercises on mastered topics
- **$\text{Var}(K)$** : Penalty for imbalanced knowledge levels across topics

- ***min(K)***: Reward for raising the weakest topic
- ***E***: Engagement level reward (keeps student motivated)
- ***P_dropout***: Large penalty (−10) when engagement reaches zero
- ***B_mastery***: Progressive mastery bonuses (+0.5 per topic > 0.5, +1.5 per topic > 0.8, +20 for full mastery)

e. Start State

Each episode begins with a simulated student initialized with random knowledge levels uniformly sampled between 0.05 and 0.25 across all five topics and a high engagement level uniformly sampled between 0.7 and 0.9, simulating a true beginner starting a tutoring session. Consecutive correct/incorrect streaks are initialized to zero, session progress starts at step 0, and the total reward accumulator is reset. This randomized initialization ensures the agent must generalize across diverse initial student profiles rather than memorizing a fixed starting configuration.

f. Terminal Conditions

- **Mastery Achieved (terminated=True)**: All 5 knowledge values reach ≥ 0.8
- **Student Dropout (truncated=True)**: Engagement drops to ≤ 0.0
- **Session Timeout (truncated=True)**: Step count reaches 200 (maximum session length)

3. System Analysis And Design

a. Deep Q-Network (DQN)

The DQN implementation uses the stable-baselines3 library, which provides a standard DQN with several key features. The network architecture is a multi-layer perceptron (MlpPolicy) with two hidden layers of 64 units each and ReLU activations. The input layer receives the 13-dimensional observation vector and the output layer produces Q-values for all 45 actions. Key architectural components include:

- **Experience Replay Buffer**: Stores past transitions (state, action, reward, next_state) and samples random mini-batches for training, breaking temporal correlations and stabilizing learning.
- **Target Network**: A separate copy of the Q-network that is updated periodically, providing stable Q-value targets and preventing oscillation during training.
- **Epsilon-Greedy Exploration**: Starts with high exploration (epsilon=1.0) and linearly decays to a final value, balancing exploration of new actions with exploitation of learned knowledge.

DQN is well-suited for this environment because it handles discrete action spaces naturally and the experience replay mechanism helps learn from the diverse range of student states encountered during tutoring sessions.

b. Policy Gradient Method ([REINFORCE/PPO])

REINFORCE: Implemented manually in PyTorch with a policy network of 2 hidden layers (configurable size: 64/128/256 units) with ReLU activations and a softmax output layer over 45 actions. The algorithm uses Monte Carlo returns—collecting full episode trajectories before updating—with an optional baseline (mean return subtraction) to reduce variance. The policy gradient is computed as: $\nabla J(\theta) = \sum_t \nabla \log \pi(a_t | s_t; \theta) (G_t - b)$, where G_t is the discounted return and b is the baseline.

PPO: Uses the stable-baselines3 implementation with an actor-critic MlpPolicy (two hidden layers of 64 units each for both the policy and value networks). PPO's key innovation is the clipped surrogate objective: $L^{\text{CLIP}}(\theta) = \min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)$, which prevents destructive large policy updates.

Advantages are computed using Generalized Advantage Estimation (GAE), and an entropy bonus encourages continued exploration.

4. Implementation

a. DQN

Ten hyperparameter configurations were tested, each trained for 150,000 timesteps. The best configuration (Run 10) achieved a mean reward of 832.52.

Learning Rate	Gamma	Replay Buffer Size	Batch Size	Exploration Strategy	Mean Reward
0.0001	0.99	50000	64	0.2 / 0.05	663.56
0.0005	0.95	10000	32	0.1 / 0.01	414.78
0.001	0.99	50000	128	0.3 / 0.10	693.13
0.005	0.9	5000	32	0.2 / 0.05	810.00
0.01	0.95	10000	64	0.1 / 0.01	781.47
0.0001	0.9	10000	128	0.3 / 0.10	668.02
0.0005	0.99	5000	64	0.2 / 0.01	791.40
0.001	0.95	50000	32	0.1 / 0.05	766.20
0.0005	0.9	10000	128	0.3 / 0.05	750.31
0.001	0.99	5000	64	0.2 / 0.10	832.52

The best DQN configuration used $lr=0.001$, $gamma=0.99$, $buffer_size=5000$, $batch_size=64$, with moderate exploration ($fraction=0.2$, $final_eps=0.10$). Higher learning rates ($0.005-0.01$) with smaller buffers generally outperformed conservative configurations, suggesting the environment benefits from faster adaptation.

b. REINFORCE

Ten configurations were tested, each trained for 3,000 episodes. The best configuration (Run 5) achieved a mean reward of 673.05.

Learning Rate	Gamma	Hidden Size	Baseline	Mean Reward
0.001	0.99	128	Yes	-135.57
0.0005	0.99	128	Yes	-136.66
0.0001	0.95	256	Yes	-138.72
0.005	0.9	64	No	562.52
0.001	0.95	256	No	673.05
0.0005	0.99	64	Yes	-6.86
0.0001	0.99	128	No	-140.69
0.005	0.95	128	Yes	-140.14
0.001	0.9	256	Yes	660.15

0.0005	0.95	64	No	95.05
--------	------	----	----	-------

A striking finding: configurations without the baseline consistently outperformed those with it (runs 4, 5, 9, 10 vs runs 1, 2, 3, 6, 7, 8). This suggests that in this environment with highly variable episode returns, the mean baseline introduces more noise than it removes. Larger hidden sizes (256) with moderate learning rates performed best.

c. PPO

Ten configurations were tested, each trained for 150,000 timesteps. The best configuration (Run 4) achieved a mean reward of 762.64.

Learning Rate	Gamma	N Steps	Batch Size	Clip Range	Ent Coef	Mean Reward
0.0003	0.99	1024	64	0.2	0.01	489.97
0.0001	0.99	2048	128	0.2	0	-141.65
0.001	0.95	512	32	0.3	0.05	660.90
0.003	0.9	256	64	0.1	0.01	762.64
0.0001	0.95	1024	32	0.3	0	-14.18
0.0003	0.99	512	128	0.1	0.05	175.27
0.001	0.99	2048	64	0.2	0.01	480.27
0.0003	0.9	256	32	0.3	0	634.69
0.0001	0.99	1024	128	0.2	0.05	-34.89
0.003	0.95	512	64	0.1	0.01	730.09

PPO performed best with higher learning rates (0.003), lower gamma (0.90), shorter rollout windows ($n_steps=256$), and a tighter clip range (0.1). Very conservative configurations ($lr=0.0001$ with large n_steps) consistently failed, indicating that this environment requires aggressive policy updates to learn effective tutoring strategies.

5. Results Discussion

Describe and interpret every visualization

a. Cumulative Rewards



Figure 1: Cumulative rewards over episodes for the best run of each method.

Figure 1 shows the cumulative reward trajectories for the best configuration of each method. DQN (left) shows the steepest and most consistent upward trajectory, indicating it accumulates reward most efficiently. REINFORCE (center) displays a slower initial phase followed by acceleration once the policy stabilizes. PPO (right) shows strong cumulative growth comparable to DQN. The consistent upward slopes across all methods confirm that all three successfully learned tutoring strategies that generate positive returns over random behavior.

Training Stability

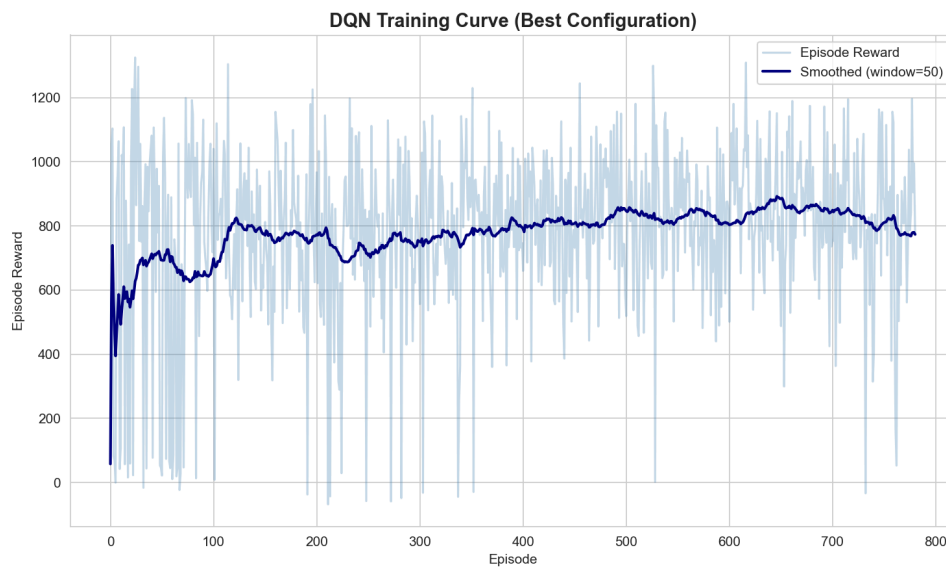


Figure 2: DQN training curve showing episode rewards and smoothed trend (window=50).

The DQN training curve (Figure 2) shows rapid initial learning in the first 200 episodes, with rewards jumping from near-zero to the 600–800 range. After episode 200, the smoothed curve stabilizes around 750–850, demonstrating convergence. The raw episode rewards show moderate variance (typical for stochastic environments with random student initial states), but the smoothed trend is remarkably stable, indicating the learned Q-values provide consistent action selection.

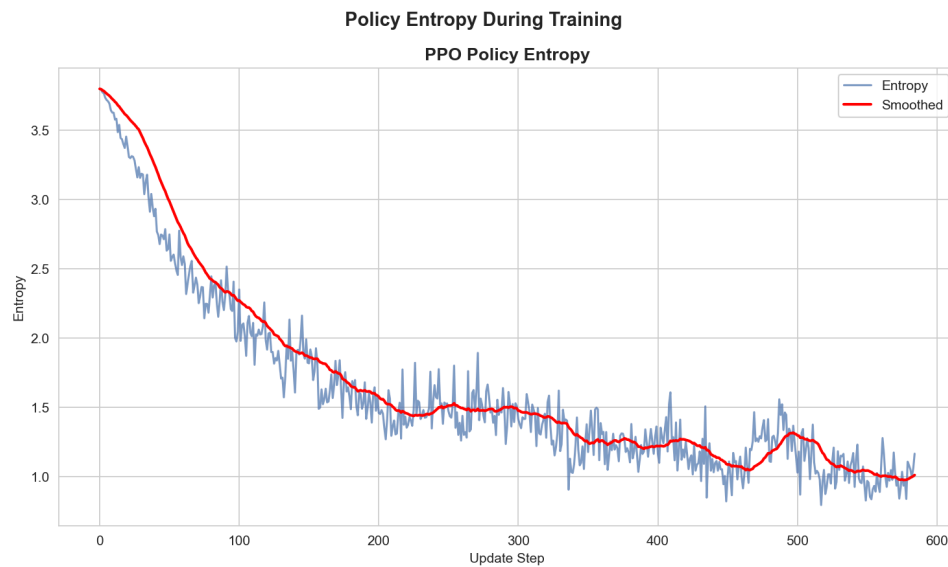


Figure 3: Policy entropy during PPO training.

Entropy starts high (the policy explores broadly) and gradually decreases as the agent converges on specific tutoring strategies. The smooth decline without sudden drops suggests PPO's clipping mechanism successfully prevents catastrophic policy changes, maintaining stable exploration throughout training.

b. Episodes To Converge

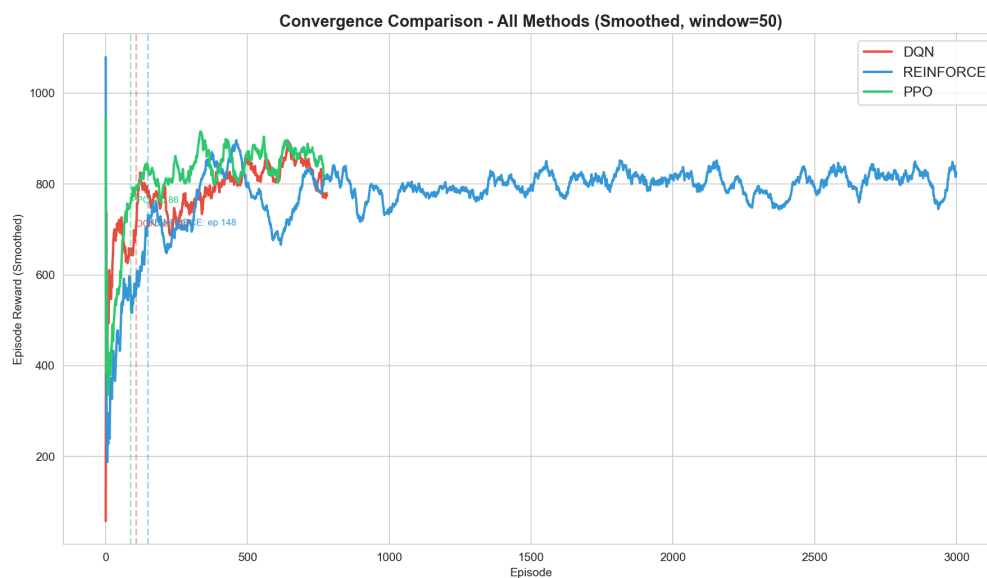


Figure 4: Convergence comparison across all three methods with smoothed episode rewards.

Figure 4 overlays the smoothed reward curves for all three methods on a single plot. Key observations:

- DQN converges fastest, reaching stable performance within approximately 150–200 episodes. The experience replay buffer allows it to reuse past experiences efficiently.
- PPO converges to a slightly lower but stable plateau, typically by episode 200–300. The clipped objective prevents performance collapses but may limit peak performance.
- REINFORCE shows the slowest convergence (500+ episodes) and highest variance, consistent with its Monte Carlo nature—it requires complete episodes before updating and lacks the stability mechanisms of DQN (replay) and PPO (clipping).

c. Generalization

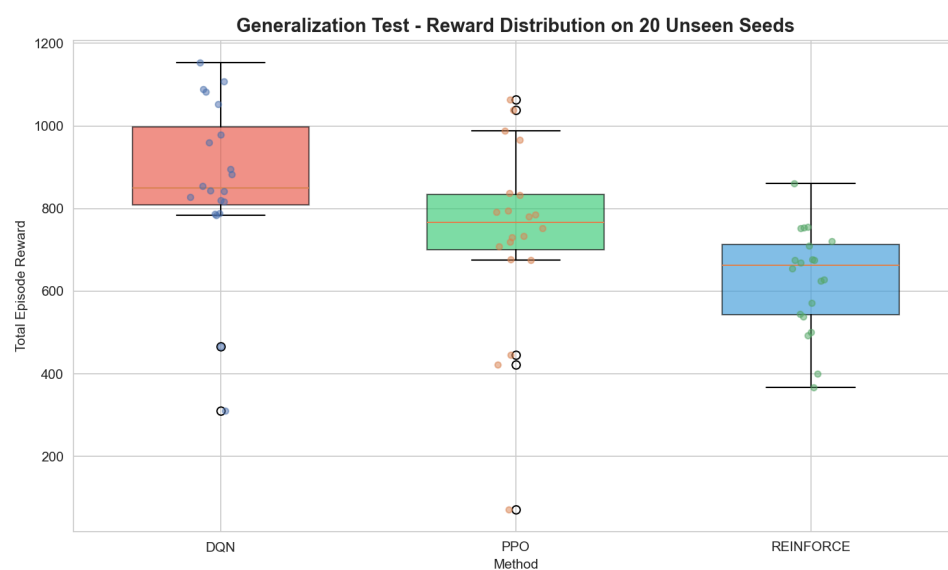


Figure 5: Reward distribution across 20 unseen seeds for each method.

Figure 5 presents box plots of total episode rewards when each trained model is tested on 20 previously unseen random seeds (different initial student knowledge and engagement levels). DQN demonstrates the highest median reward with a relatively tight interquartile range, indicating strong and consistent generalization. PPO shows competitive median performance but with wider variance, suggesting it is more sensitive to initial conditions. REINFORCE generalizes reasonably but with the largest spread, reflecting the inherent variance of its Monte Carlo policy. Crucially, all three methods significantly outperform random action selection (which averages around -30 and consistently causes student dropout), confirming that each algorithm learned genuine, transferable tutoring strategies rather than overfitting to training seeds.

6. Conclusion and Discussion

This project demonstrated that reinforcement learning can effectively learn adaptive tutoring strategies for a simulated beginner programmer. The comparison of three algorithms yields clear insights:

DQN achieved the best overall performance (mean reward: 832.52), converging fastest and generalizing most reliably. Its experience replay buffer is particularly well-suited for this environment because tutoring interactions generate diverse state transitions that benefit from being revisited during training. The off-policy nature of DQN also means it can learn from suboptimal past actions, accelerating convergence.

PPO was the second-best performer (mean reward: 762.64), offering more stable training than REINFORCE thanks to its clipped surrogate objective. However, its on-policy nature means it cannot reuse past experience, requiring more environment interactions to achieve comparable performance. PPO's entropy regularization helped maintain exploration, which is valuable in an environment where the student state evolves continuously.

REINFORCE ranked third (mean reward: 673.05) but still learned effective strategies. Its main limitation is high variance from Monte Carlo returns—in a 200-step tutoring session, the full-episode return signal is noisy because student outcomes are stochastic. Interestingly, configurations without baseline subtraction outperformed those with it, suggesting the baseline was introducing bias in this particular environment.

All three methods successfully learned the core tutoring skill: using easy exercises with support to build initial knowledge, gradually reducing support as the student improves, cycling across all five topics to maintain balanced progress, and monitoring engagement to avoid student dropout.

Improvements with additional time/resources: (1) Replace the simulated student with a model trained on real student interaction data to validate the learned policies. (2) Increase training duration and use more granular hyperparameter searches (e.g., Bayesian optimization). (3) Implement curriculum learning where the RL agent is first trained on simple student models before progressing to more complex ones. (4) Add more observation features such as time spent per exercise, historical topic preferences, and learning style indicators. (5) Explore multi-agent approaches where separate agents specialize in different difficulty levels.